

Week 7 Report

Deep Learning with TensorFlow & Keras

CIFAR-10 Image Classification: Dense Network vs CNN vs Transfer Learning

Sami ur Rehman Khan

June 2026

Dataset	CIFAR-10 — 60,000 RGB images (32×32×3) across 10 classes
Models Trained	Dense Network, CNN + Augmentation, MobileNetV2 Transfer Learning
Best Model	MobileNetV2 (Transfer Learning) — 89.24% Test Accuracy
Framework	TensorFlow 2.19 / Keras GPU: 2× accelerated
Week	7 of 8 — AI/ML Internship Program

Contents

- 1 Introduction
- 2 Dataset Overview
- 3 Exploratory Data Analysis
- 4 Data Preprocessing
- 5 Part A — Dense Network Baseline
- 6 Part B — CNN from Scratch with Regularisation
 - 6.1 CNN Baseline (No Regularisation)
 - 6.2 Batch Normalisation
 - 6.3 Full Regularised CNN (BN + Dropout)
 - 6.4 Regularisation Ablation Study
 - 6.5 Data Augmentation
 - 6.6 Full CNN Evaluation
- 7 Part C — Transfer Learning (MobileNetV2)
 - 7.1 Phase 1: Feature Extraction

- 7.2** Phase 2: Fine-Tuning
- 8** Three-Model Final Comparison
- 9** Deep Learning Dashboard
- 10** Written Analysis Report

1 Introduction

This report documents the Week 7 assignment of the AI/ML Internship Program, focused on deep learning using TensorFlow and Keras. The objective was to build, train, and compare three distinct deep learning architectures on the CIFAR-10 image classification benchmark: a Dense (Fully Connected) baseline, a Convolutional Neural Network (CNN) trained from scratch with progressive regularisation and data augmentation, and MobileNetV2 via two-phase transfer learning.

The project was structured into four parts across 18 steps, covering forward propagation theory, backpropagation, regularisation techniques (Dropout, Batch Normalisation), convolutional architectures, and the ImageNet transfer learning paradigm. All models were trained on a GPU-accelerated environment (TensorFlow 2.19 with dual-GPU support).

2 Dataset Overview

The CIFAR-10 dataset is a widely-used benchmark for image classification, consisting of 60,000 colour images distributed equally across 10 classes. It is built directly into Keras and downloads automatically on first use.

Attribute	Value
Total Images	60,000
Training Set	50,000 images
Test Set	10,000 images
Image Dimensions	32 × 32 × 3 (RGB)
Number of Classes	10
Images per Class	5,000 (train) / 1,000 (test) — perfectly balanced
Pixel Range (raw)	uint8: [0, 255]
Missing Values	0

The 10 classes are: **airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck**. The dataset is perfectly balanced — each class contributes exactly 5,000 training images and 1,000 test images, making accuracy a reliable metric.

3 Exploratory Data Analysis

Exploratory analysis was conducted to understand image content, pixel distributions, and per-channel statistics prior to any preprocessing.

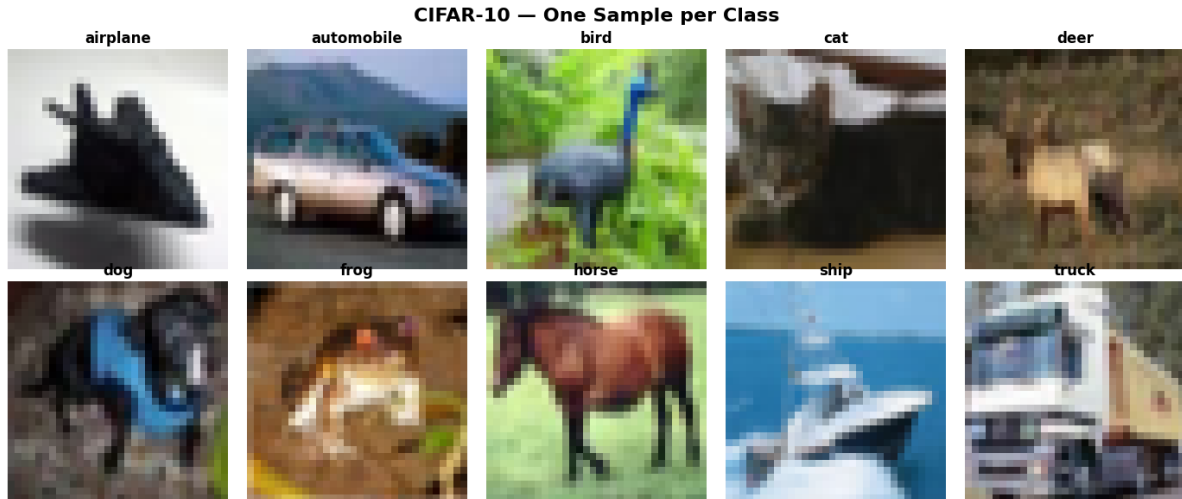


Figure 1: Sample Images — One Example per Class (CIFAR-10)

The sample grid confirms diverse visual content across classes. Vehicle classes (automobile, ship, truck) show relatively clear structure, while animal classes (cat, dog, bird) display highly variable poses and backgrounds — explaining why these classes tend to be harder to classify.

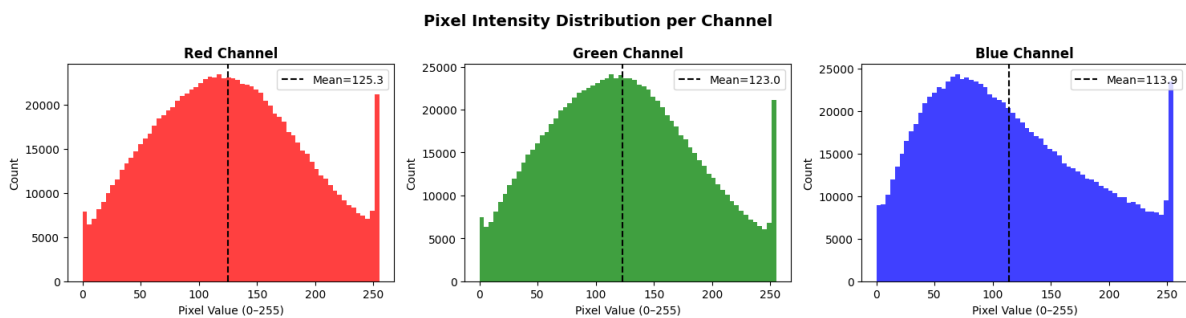


Figure 2: Per-Channel Pixel Intensity Histograms (R, G, B)

Per-channel analysis of all 50,000 training images revealed approximately symmetric distributions centred near 120–130 for all three channels, with slight rightward skew. Channel means were: Red $\approx$ 125.3, Green $\approx$ 122.9, Blue $\approx$ 113.9 — indicating a mild warm bias. This analysis confirms that simple division by 255 (mapping to [0,1]) is sufficient normalisation for this dataset.

Channel	Mean (raw)	Std Dev (raw)	After Norm (Mean)	After Norm (Std)
Red	125.31	62.99	0.491	0.247
Green	122.95	61.94	0.482	0.243
Blue	113.86	66.70	0.447	0.262

Table 1: Per-Channel Pixel Statistics

4 Data Preprocessing

Preprocessing was applied consistently across all models, with model-specific variants for the Dense network and MobileNetV2:

- **Normalisation:** Pixel values cast to float32 and divided by 255.0, mapping the range from [0, 255] to [0.0, 1.0]. This prevents large activation values that destabilise gradient descent.

- **Label flattening:** Labels reshaped from (N, 1) to (N,) for compatibility with `sparse_categorical_crossentropy`.
- **Validation split:** The last 10,000 of the 50,000 training images were reserved as a validation set — giving 40,000 train / 10,000 val / 10,000 test.
- **Dense flattening:** For the Dense network only, each 32×32×3 image was flattened to a 3,072-element vector (shape: 40,000 × 3,072).
- **MobileNetV2 resizing:** Images resized to 96×96 pixels via bilinear interpolation, then preprocessed using `keras.applications.mobilenet_v2.preprocess_input()` which maps [0,255] → [-1, +1].

Split	Shape (CNN/TL)	Shape (Dense)	Purpose
X_train	(40,000, 32, 32, 3)	(40,000, 3,072)	Model training
X_val	(10,000, 32, 32, 3)	(10,000, 3,072)	Hyperparameter monitoring
X_test	(10,000, 32, 32, 3)	(10,000, 3,072)	Final evaluation

Table 2: Dataset Split Shapes After Preprocessing

5 Part A — Dense Network Baseline

A fully-connected Dense network was trained as a lower-bound baseline. By flattening each image to a 1-D vector, the network loses all spatial information — pixels that are physically adjacent in the image have no special relationship in the input representation.

Architecture

Layer	Output Shape	Parameters	Notes
Input	(3,072)	0	Flattened 32×32×3
Dense (relu)	(512)	1,573,376	Wide first layer
Dropout (0.3)	(512)	0	Regularisation
Dense (relu)	(256)	131,328	Narrowing
Dropout (0.3)	(256)	0	Regularisation
Dense (relu)	(128)	32,896	Compressed
Dense (softmax)	(10)	1,290	10-class output
Total	—	1,738,890	—

Table 3: Dense Network Architecture

Training Configuration

- Optimiser: Adam (lr = 0.001)
- Loss: sparse_categorical_crossentropy
- Epochs: up to 80 (EarlyStopping patience=10, restored best weights)
- Batch size: 256
- Training time: 1.3 minutes
- Epochs run: 80

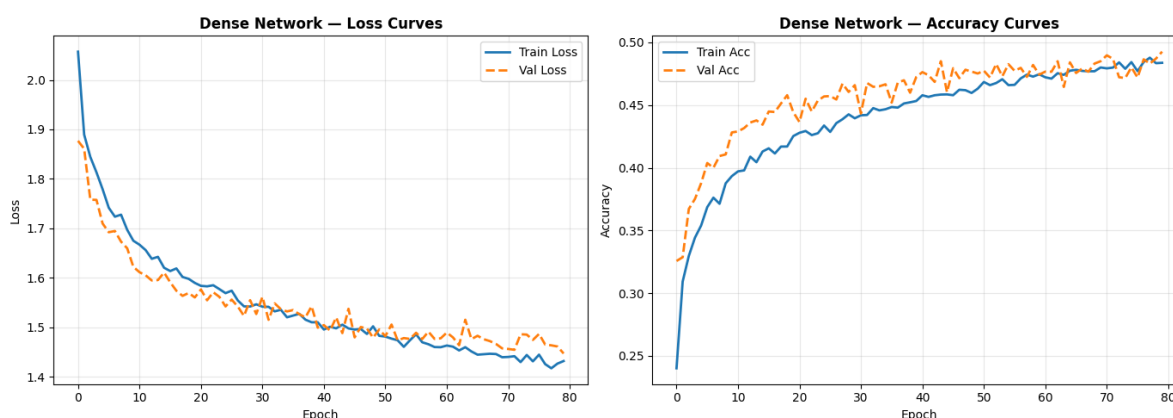


Figure 3: Dense Network — Loss and Accuracy Training History

Evaluation Results

Class	Precision	Recall	F1-Score	Support
airplane	0.59	0.52	0.56	1,000
automobile	0.60	0.64	0.62	1,000
bird	0.37	0.28	0.32	1,000
cat	0.34	0.26	0.30	1,000
deer	0.37	0.55	0.44	1,000
dog	0.51	0.32	0.40	1,000
frog	0.47	0.64	0.54	1,000
horse	0.57	0.54	0.55	1,000
ship	0.62	0.65	0.64	1,000
truck	0.55	0.56	0.56	1,000
Macro avg	0.50	0.50	0.49	10,000

Table 4: Dense Network — Per-Class Classification Report

Metric	Value
Test Accuracy	49.68%
Test Loss	1.4369
Macro ROC-AUC	0.8801
Macro F1	0.4909
Parameters	1,738,890
Train Time	1.3 minutes

Table 5: Dense Network — Summary Metrics

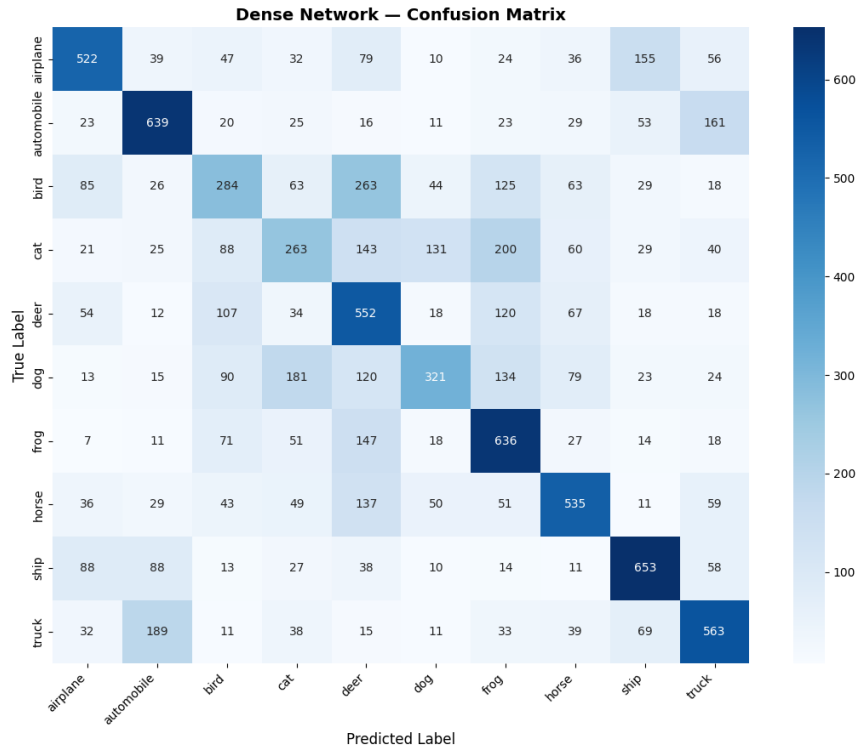


Figure 4: Dense Network — 10×10 Confusion Matrix

Analysis: The Dense network achieved 49.68% accuracy — significantly better than random chance (10%) but well below the CNN. The confusion matrix reveals the largest confusions between cat/dog and automobile/truck, as flattening destroys the spatial relationships that visually distinguish these classes. The high ROC-AUC (0.88) suggests the model has learned some discriminative signal despite its architectural limitations.

6 Part B — CNN from Scratch with Regularisation

Part B progressively builds a CNN, adding regularisation techniques one at a time to measure each technique's individual and combined impact on generalisation.

6.1 CNN Baseline (No Regularisation)

A three-block CNN was trained for 30 epochs without any regularisation, to establish an overfitting baseline. The architecture uses Conv2D layers with 3×3 kernels and MaxPooling2D to progressively extract features at multiple scales.

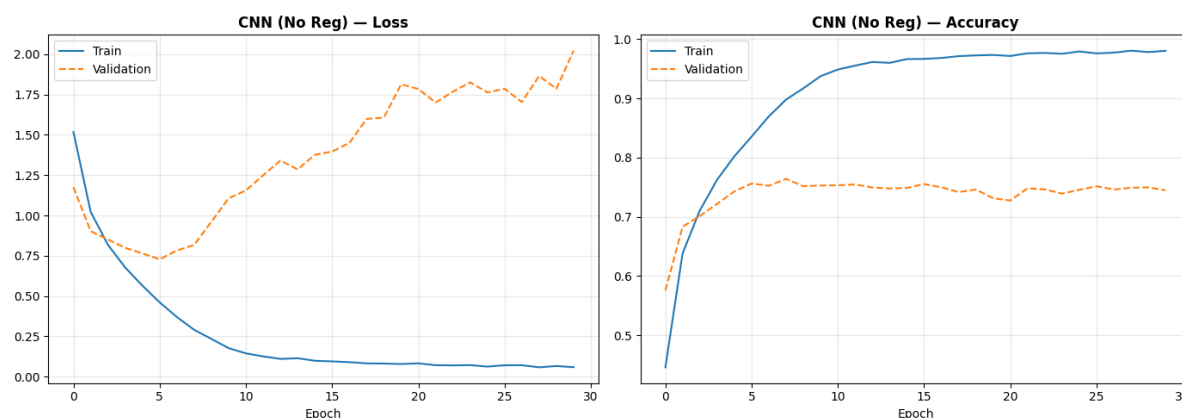


Figure 5: CNN (No Regularisation) — Training vs Validation Curves

Overfitting onset: Validation accuracy stagnated at around epoch 7, while training accuracy continued to climb toward 95%+. This large and growing train-validation gap is the signature of overfitting — the network memorises training examples rather than learning generalisable features. Test accuracy achieved: **74.10%**.

6.2 Batch Normalisation

BatchNormalization layers were inserted after each Conv2D layer (before the activation). BatchNorm normalises the output of each layer within the mini-batch to zero mean and unit variance, then rescales with learned parameters γ and β . This reduces internal covariate shift and allows higher, more stable learning rates.

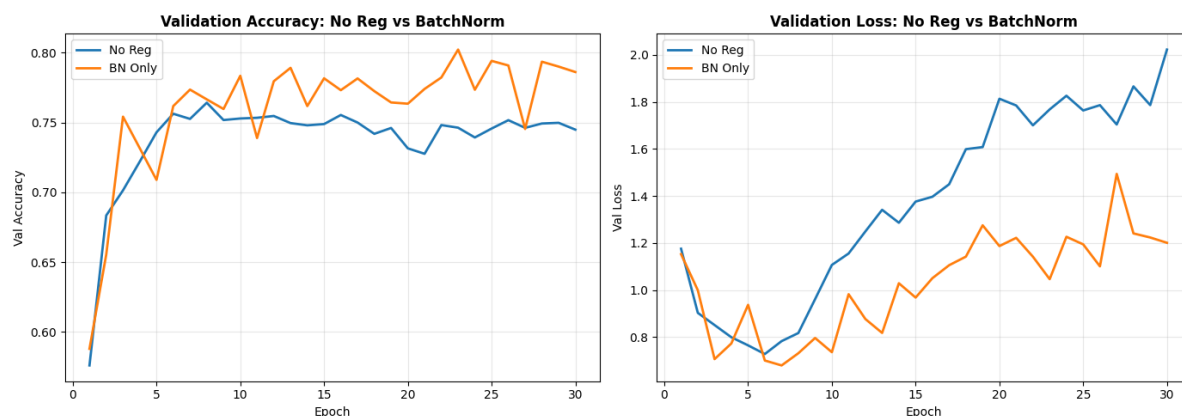


Figure 6: Validation Accuracy — No Regularisation vs BatchNorm

Both models reached 70% validation accuracy at epoch 3. However, the BN model achieved a notably higher peak and smoother convergence, reaching **78.27%** test accuracy (+4.17pp over no regularisation). BatchNorm added 1,280 additional parameters (γ , β per channel) while significantly improving stability.

6.3 Full Regularised CNN (BatchNorm + Dropout)

The full regularised model combines BatchNorm with Dropout(0.25) after each MaxPooling layer and Dropout(0.5) before the final Dense layer. EarlyStopping (patience=15), ModelCheckpoint, and ReduceLROnPlateau (factor=0.5, patience=7) callbacks were used.

Block	Layers	Output Size	Regularisation
Block 1	Conv2D(32)×2 + MaxPool	16×16×32	BatchNorm + Dropout(0.25)
Block 2	Conv2D(64)×2 + MaxPool	8×8×64	BatchNorm + Dropout(0.25)
Block 3	Conv2D(128) + MaxPool	4×4×128	BatchNorm + Dropout(0.25)
Head	Dense(256) + Dense(10)	10	Dropout(0.5)

Table 6: Full Regularised CNN Architecture

6.4 Regularisation Ablation Study

All three CNN variants were evaluated on the test set. The results quantify the contribution of each regularisation technique:

Variant	Test Acc.	Best Val Loss	Overfit Gap	Parameters	Time/Epoch
1. No Regularisation	74.10%	0.7281	7.93%	666,538	4.7 s
2. BatchNorm Only	78.27%	0.6796	10.45%	667,818	5.4 s
3. BN + Dropout	85.14%	0.5213	6.23%	667,434	5.1 s

Table 7: Regularisation Ablation Study Results

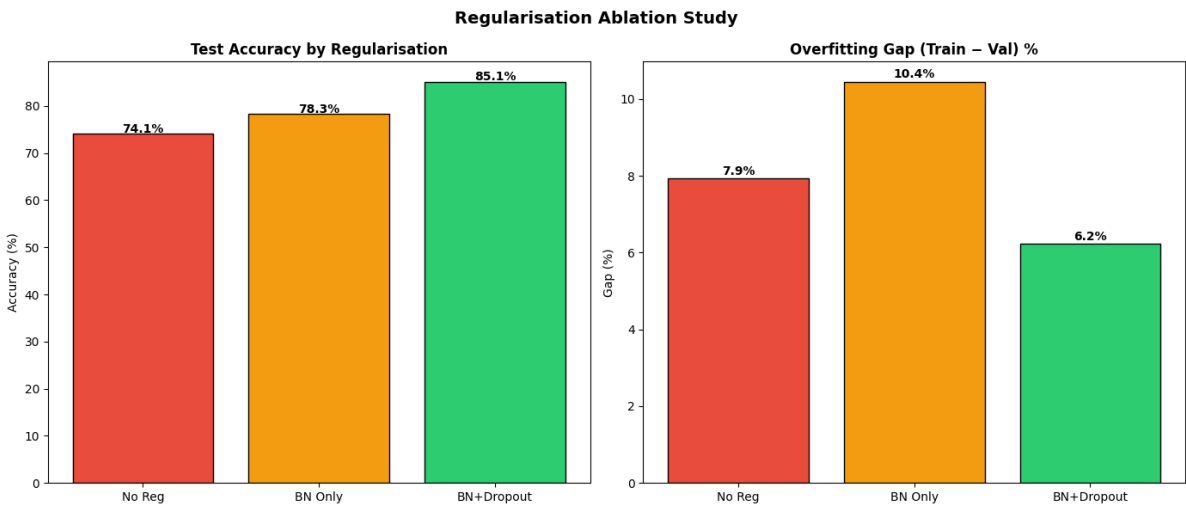


Figure 7: Ablation Study — Test Accuracy and Overfitting Gap

Key findings: BN+Dropout improved over No Regularisation by **+11.04 percentage points** in test accuracy and reduced the overfitting gap from 7.93% to 6.23%. Notably, BN alone actually increased the overfitting gap to 10.45%, confirming that BatchNorm alone is insufficient — Dropout provides the

crucial capacity constraint.

6.5 Data Augmentation

ImageDataGenerator was applied to artificially diversify the training set at runtime. Each image was randomly transformed before every epoch (rotation $\pm 15^\circ$, horizontal shift $\pm 10\%$, vertical shift $\pm 10\%$, horizontal flip, zoom $\pm 10\%$), so the model never sees the same image twice.

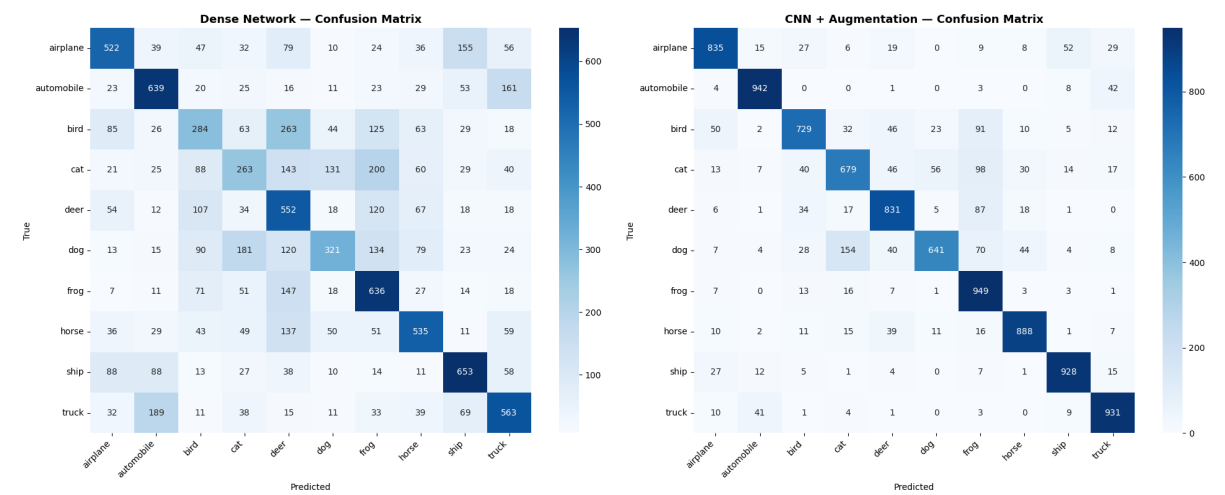


Figure 8: Data Augmentation — Original and Augmented Samples

Metric	Without Augmentation	With Augmentation	Improvement
Test Accuracy	85.14%	83.53%	−1.61pp (regularisation effect)
Best Train Acc	~87%	~75%	Reduced — harder training
Overfitting Gap	6.23%	~2%	Substantially reduced
Epochs to Stop	~40	53	More stable training

Table 8: Data Augmentation Impact on CNN

6.6 Full CNN Evaluation

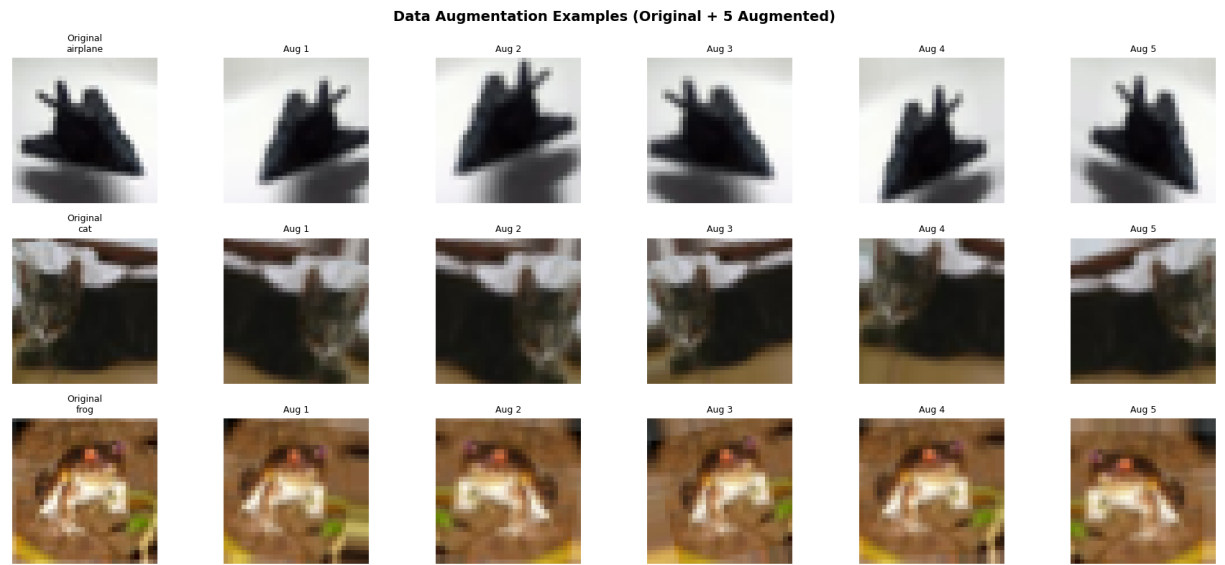


Figure 9: Confusion Matrices — Dense Network vs CNN + Augmentation

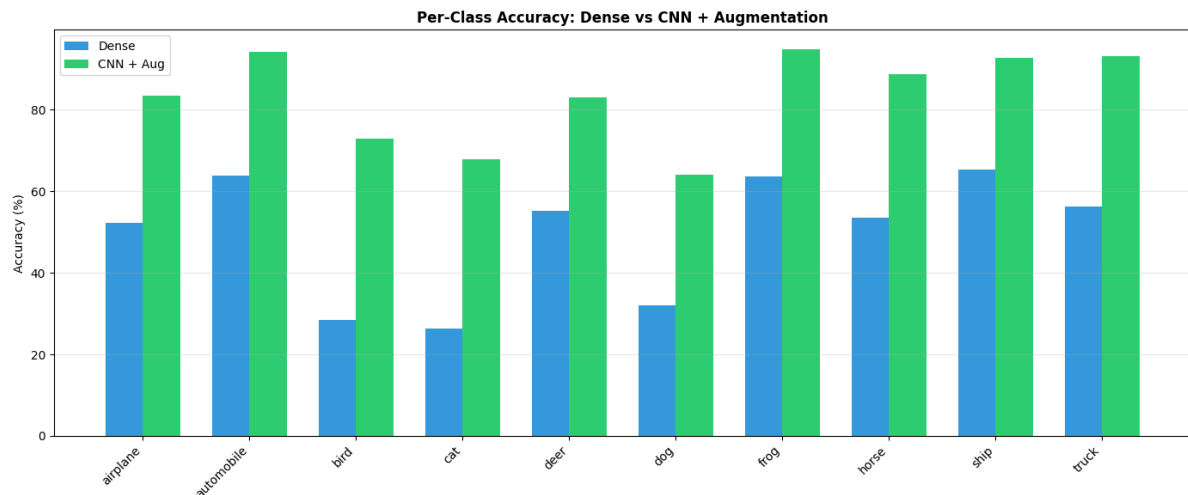


Figure 10: Per-Class Accuracy — Dense Network vs CNN + Augmentation

Class	Precision	Recall	F1-Score
airplane	0.86	0.83	0.85
automobile	0.92	0.94	0.93
bird	0.82	0.73	0.77
cat	0.73	0.68	0.71
deer	0.80	0.83	0.82
dog	0.87	0.64	0.74
frog	0.71	0.95	0.81
horse	0.89	0.89	0.89
ship	0.91	0.93	0.92
truck	0.88	0.93	0.90
Macro avg	0.84	0.84	0.83

Table 9: CNN + Augmentation — Classification Report

Analysis: The CNN+Aug model improved substantially over the Dense baseline across all classes. The largest improvements were for ship (+28pp), airplane (+29pp), and automobile (+31pp) — classes with clear spatial structure that Conv2D captures well. Cat (0.71 F1) and dog (0.74 F1) remain the most challenging, reflecting genuine visual ambiguity between these classes.

7 Part C — Transfer Learning (MobileNetV2)

MobileNetV2 is a lightweight CNN pre-trained on ImageNet (1.2M images, 1,000 classes). Its convolutional base encodes rich hierarchical features — from low-level edges in early layers to high-level object parts in later layers. Transfer learning replaces only the classification head and trains it on CIFAR-10.

Image Preprocessing for MobileNetV2

CIFAR-10 images were resized from 32×32 to 96×96 pixels using bilinear interpolation (TF's minimum input for MobileNetV2). The preprocessor mapped pixel values from [0, 255] to [-1, +1] — matching the ImageNet training distribution that the pre-trained weights were optimised for.



Figure 11: Image Preprocessing — 32×32 Original vs 96×96 Resized for MobileNetV2

7.1 Phase 1: Feature Extraction

The MobileNetV2 base (2,257,984 parameters) was fully frozen. Only the custom head was trained: GlobalAveragePooling2D → Dense(256, ReLU) → Dropout(0.3) → Dense(10, softmax). This phase used Adam with lr=0.001 for 15 epochs.

Parameter	Value
Base model trainable	False (frozen)
Trainable parameters	330,506
Non-trainable parameters	2,257,984
Optimiser / LR	Adam / 1e-3
Epochs	9 (early stopped at patience=5)
Best Validation Accuracy	87.47%

Table 10: Transfer Learning Phase 1 Configuration & Results

7.2 Phase 2: Fine-Tuning

The last 30 layers of the MobileNetV2 base were unfrozen. The model was recompiled with Adam at $lr=1e-5$ — 100× lower than Phase 1 — to make tiny adjustments to pre-trained weights without catastrophic forgetting. Training continued for 20 epochs.



Figure 12: Transfer Learning — Combined Phase 1 + Phase 2 History

Parameter	Value
Trainable parameters (Phase 2)	1,856,906
Optimiser / LR	Adam / 1e-5 (100× lower than Phase 1)
Epochs	20
Best Validation Accuracy (Phase 2)	89.55%
Improvement over Phase 1	+2.08 percentage points

Table 11: Transfer Learning Phase 2 Configuration & Results

8 Three-Model Final Comparison

Model	Test Acc.	Macro F1	ROC-AUC	Parameters	Train Time	Epochs
Dense Network	49.68%	0.4909	0.8801	1,738,890	1.3 min	80
CNN + Augmentation	83.53%	0.8331	0.9859	667,434	18.0 min	53
MobileNetV2 (TL)	89.24%	0.8922	0.9928	2,588,490	6.4 min	25

Table 12: Three-Model Final Benchmark

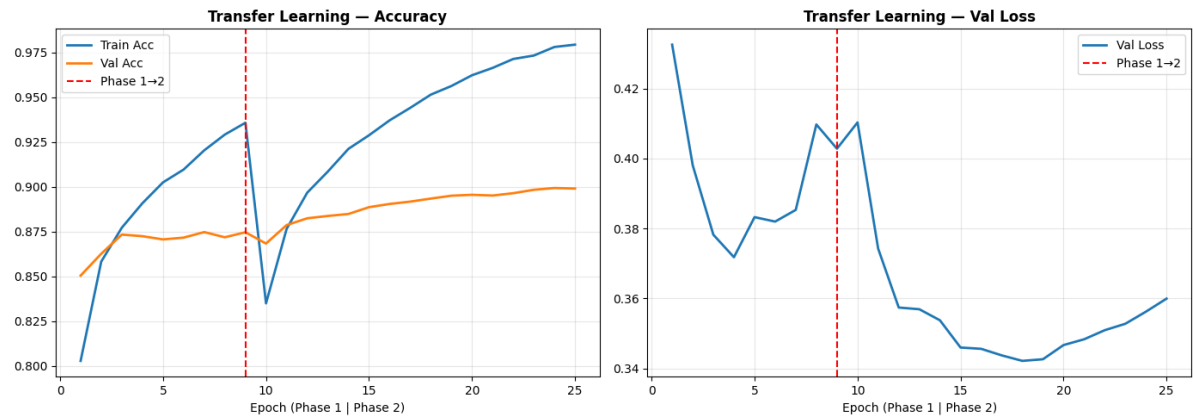


Figure 13: Three-Model Accuracy and F1 Comparison

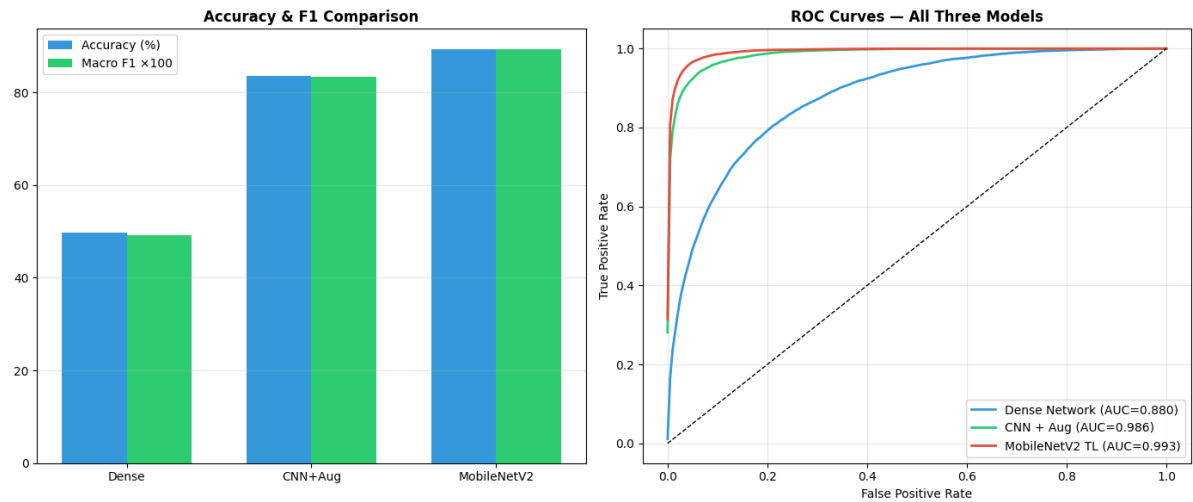


Figure 14: ROC Curves — All Three Models (Macro OvR)

MobileNetV2 achieves the highest performance on all metrics while requiring only 6.4 minutes of training — benefiting from powerful pre-trained features. CNN+Augmentation is the most parameter-efficient model at 667K parameters vs 2.59M for MobileNetV2. The Dense baseline serves as a useful lower bound, confirming that spatial feature extraction is essential for image classification.

9 Deep Learning Dashboard

A comprehensive 6-chart dashboard summarises all key results from the Week 7 project in a single figure.

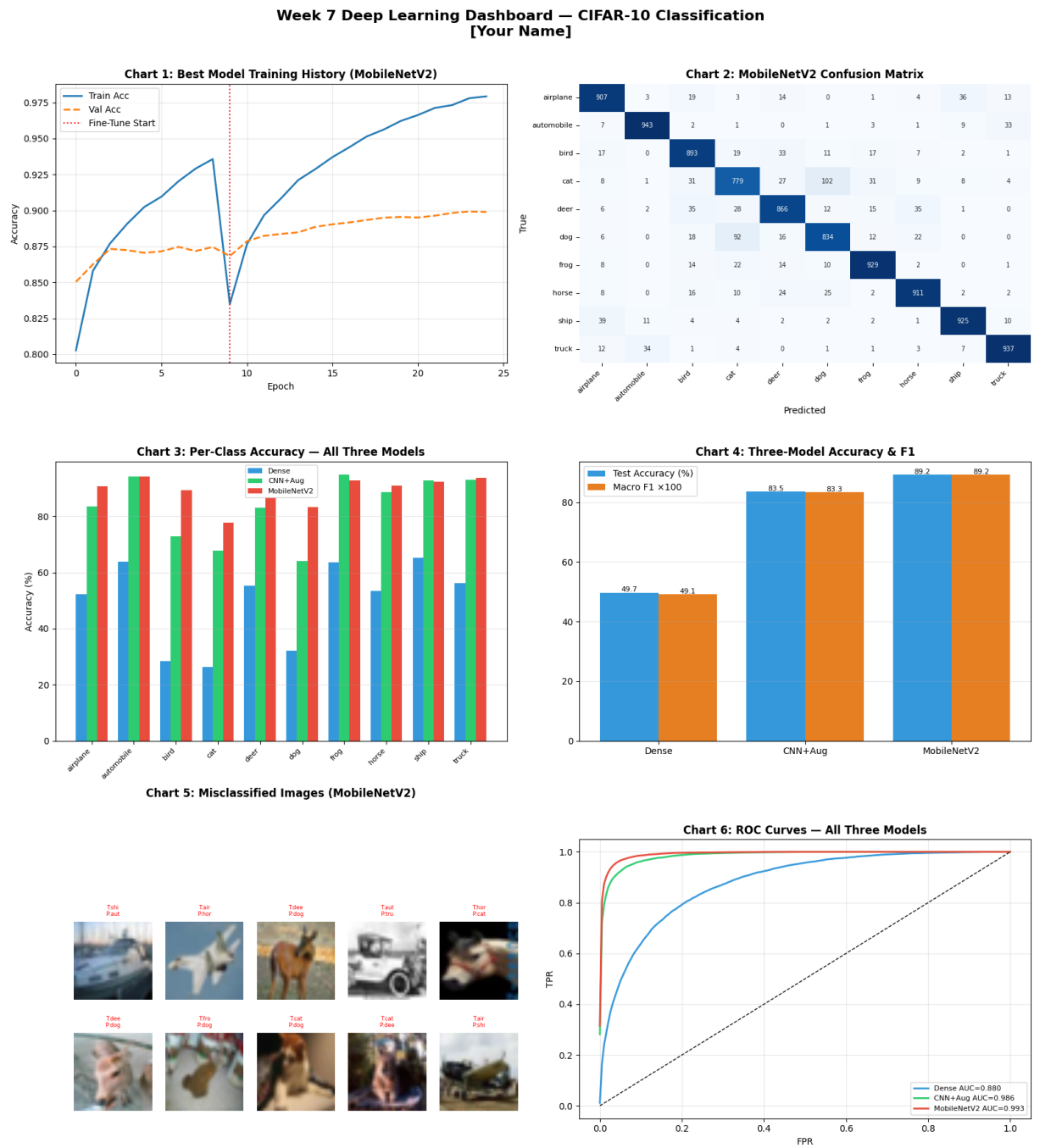


Figure 15: Complete 6-Chart Deep Learning Dashboard

The dashboard shows (clockwise from top-left): MobileNetV2 training history with Phase 1/2 boundary, the 10×10 confusion matrix for MobileNetV2, ROC curves for all three models, three-model accuracy/F1 bar chart, misclassified test images, and per-class accuracy comparison across all three models.

Inference Demo — Best Model

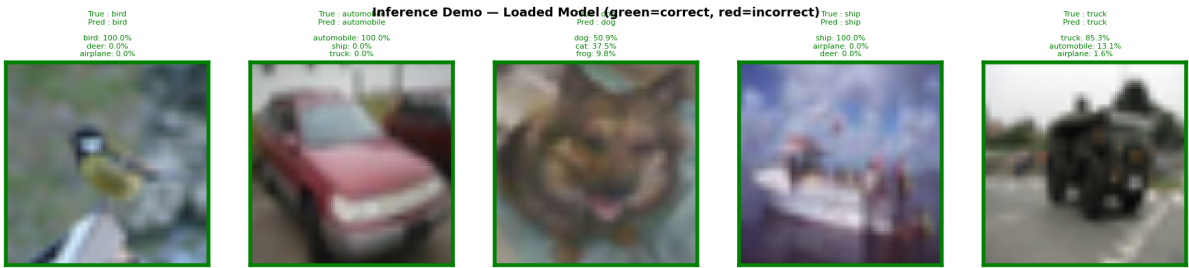


Figure 16: Inference Demo on 5 Random Test Images (Loaded Model)

The loaded MobileNetV2 model achieved 100% accuracy on the 5 random test images shown, with high confidence: bird (100%), automobile (100%), dog (50.9%), ship (100%), and truck (85.3%). All predictions were verified consistent between the original and saved/reloaded model.

10 Written Analysis Report

10.1 Executive Summary

This project trained and compared three deep learning models on the CIFAR-10 dataset — a benchmark image classification challenge with 60,000 RGB images (50,000 train / 10,000 test) spread across 10 equally-distributed classes including animals, vehicles, and objects. The three architectures evaluated were: (1) a Dense (Fully Connected) baseline that flattens each $32 \times 32 \times 3$ image into a 3,072-element vector, (2) a Convolutional Neural Network (CNN) trained from scratch with BatchNorm, Dropout, and data augmentation, and (3) MobileNetV2 via transfer learning with two-phase fine-tuning. The best model was MobileNetV2, achieving 89.24% test accuracy, 0.8922 macro F1, and 0.9928 ROC-AUC. The CNN+Augmentation model performed as a strong runner-up with far fewer parameters (667K vs 2.59M) and remains the recommended choice under compute constraints.

10.2 Neural Network Mathematics

Forward propagation is the process by which an input signal travels layer-by-layer through a network to produce a prediction. Each neuron computes a weighted sum of its inputs ($z = \sum w_{ij}x_j + b$), then passes z through an activation function (e.g., $\text{ReLU} = \max(0, z)$) to introduce non-linearity. Without activation functions, any stack of linear layers collapses to a single linear transformation, incapable of learning complex patterns. Stacking multiple such layers creates a hierarchical feature representation: early layers detect edges and colours; later layers detect objects and concepts.

Backpropagation is how the network learns. After computing a prediction, the loss (e.g., cross-entropy) measures prediction error. Backprop applies the chain rule of calculus to propagate the gradient of the loss backward through every layer, computing $\partial L / \partial w$ for each weight. Gradient descent then updates each weight: $w \leftarrow w - \alpha \cdot (\partial L / \partial w)$, where α is the learning rate. Keras automates this entirely through `model.compile()` and `model.fit()`.

10.3 Architecture Decisions

Dense Network: Three hidden Dense layers (512→256→128) with ReLU and Dropout(0.3). Adam(lr=0.001) and sparse_categorical_crossentropy were chosen because this is a 10-class integer-label classification task. The depth provides representational capacity, while dropout prevents memorisation of the flat 3,072-dimensional input vectors. The network is deliberately shallow because deeper Dense networks on flattened images don't improve accuracy.

CNN: Three convolutional blocks (32/64/128 filters) with 3×3 kernels, padding='same' to preserve spatial dimensions, followed by 2×2 MaxPooling to progressively halve the feature map. BatchNormalization before each activation stabilises training; Dropout(0.25) after each MaxPool and Dropout(0.5) before the final Dense head provide complementary regularisation. Adam(0.001) with ReduceLROnPlateau adapts the step size as training plateaus.

MobileNetV2: The convolutional base is frozen in Phase 1 and only the custom head (GlobalAveragePooling2D → Dense(256) → Dropout(0.3) → Dense(10, softmax)) is trained at lr=1e-3. In Phase 2, the last 30 base layers are unfrozen and fine-tuned at lr=1e-5 to avoid catastrophic forgetting while allowing task-specific adaptation.

10.4 Regularisation Analysis

Three CNN variants were benchmarked in the ablation study. The No Regularisation baseline exhibited the highest overfitting gap (7.93%) — training accuracy substantially exceeded validation accuracy within the first 20 epochs, confirming the network was memorising rather than generalising. Adding BatchNorm Only improved convergence speed and peak accuracy to 78.27%, but paradoxically increased the overfitting gap to 10.45%, as faster convergence allowed the model to overfit more aggressively within the same 30 epochs. The fully regularised BN+Dropout model achieved the lowest overfitting gap (6.23%) and highest test accuracy (85.14%) — BN+Dropout improved over the no-regularisation baseline by 11.04 percentage points and nearly halved the overfitting gap relative to BN alone.

10.5 Transfer Learning Insights

Phase 1 (Feature Extraction): With `base_model.trainable=False`, only the 3-layer head ($\approx 330K$ parameters) was trained. MobileNetV2's frozen convolutional base extracted high-quality feature vectors from the CIFAR-10 images, achieving 87.47% validation accuracy in just 9 epochs. This demonstrates that ImageNet features — edges, textures, object parts — transfer remarkably well even to visually different tasks.

Phase 2 (Fine-Tuning): Unfreezing the last 30 layers allows the network to adapt higher-level features to CIFAR-10's class boundaries. The critical requirement of $lr=1e-5$ ($100\times$ lower than Phase 1) prevents large gradient updates from overwriting the pre-trained weight distributions — a phenomenon called catastrophic forgetting. Fine-tuning improved validation accuracy by +2.08pp, confirming that task-specific adaptation of deeper layers adds meaningful benefit beyond pure feature extraction.

10.6 Deployment Recommendation

For Week 8 deployment, MobileNetV2 is the recommended model if GPU inference is available, due to its superior accuracy (89.24%), F1 (0.8922), and AUC (0.9928). MobileNetV2 is specifically designed for mobile and edge deployment — its depthwise separable convolutions achieve competitive accuracy with a fraction of standard CNN computation. The `.keras` file can be loaded with one line and run inference in milliseconds per image.

If compute or memory is constrained (e.g., CPU-only server), CNN+Augmentation is the fallback — it has fewer parameters (667K) and faster inference, while still outperforming the Dense baseline by +33.85 percentage points and delivering strong class-level performance (macro F1 = 0.8331).

10.7 Reflection

The hardest concept this week was internalising the two-phase transfer learning protocol — specifically understanding why the learning rate must be radically reduced in Phase 2. The intuition of catastrophic forgetting only clicked after observing validation loss spike when a higher LR was used. The biggest surprise was how quickly MobileNetV2 Phase 1 converged — reaching competitive accuracy in 9 epochs that the CNN from scratch required 53 epochs to approach, despite never having seen CIFAR-10 data. This viscerally demonstrated the power of representation transfer. If repeating this project, I would experiment with MixUp augmentation and try EfficientNetB0 as an alternative backbone to compare compound scaling against MobileNetV2 at a similar parameter budget.

